TRACE Demo Dashboard

1. What this demo is

This is a small working demo I built to show how I would approach the monitoring side of the TRACE project. It simulates a POAO (Products of Animal Origin) cold-chain journey, for example a truck of dairy or beef travelling from a farm in Northern Ireland across the border into Ireland, and shows how IoT sensors and hyperspectral imaging could be used to monitor it, flag problems, and feed into reporting.

It is not a claim that this is how TRACE will actually be built. There is no real project data yet, so I generated realistic synthetic data myself, then built a simple, transparent monitoring pipeline and an interactive dashboard on top of it. The goal is to demonstrate that I understand the technologies named in the job description (IoT sensors, hyperspectral imaging, computer vision) and that I can turn that into something practical: documentation, dashboards, alerts, and a risk register, which is what the role actually asks for.

> *Key point to make early in the interview: this is a synthetic demo built to show my approach and technical understanding, not real TRACE data or a finished system.*

2. The data: what I simulated and why

Since TRACE hasn't started collecting real pilot data yet, I built a synthetic dataset that mimics what the project would realistically produce:

- 6 fictional cross-border SMEs, a beef producer, two dairies, a poultry farm, a lamb producer, and a seafood company, each with routes crossing between Northern Ireland and Ireland.
- 42 shipments in total, each one a 48-hour journey (e.g. farm/processing → transport → retail).
- IoT sensor readings every 15 minutes: temperature, humidity, how often the vehicle door opens, and vibration (a proxy for rough handling).
- A hyperspectral imaging camera capturing a picture of the product at 4 physical checkpoints per journey (Farm/Origin, Processing, In-Transit spot check, Retail/Arrival). This is separate from the continuous sensors, because in real life you don't carry a hyperspectral camera everywhere, you use it at hand-off points.
- A mix of scenarios: most shipments are normal, some have a genuine cold-chain breach (like a fridge unit failing), some have small temperature wobbles, and a few have a faulty sensor (to show what a data-quality problem looks like, as opposed to a real product problem).

This mix matters for the interview: it shows I thought about realistic failure modes, not just a clean happy-path dataset.

3. The dataset in detail

This section is here so I can answer any "what's actually in the data" question with specifics, rather than talking in generalities. The demo produces four files, and they build on each other.

File 1: shipments.csv, the shipment summary table

One row per shipment, 42 rows in total. This is the table a project manager would glance at first.

- shipment_id: a unique code for the journey, like SHP-0008.
- sme_id and sme_name: which of the 6 companies the shipment belongs to.
- product: beef, dairy, poultry, lamb, or seafood.
- route: the cross-border journey, for example Ballymena to Dublin.
- scenario: the storyline behind the shipment. Normal, cold_chain_breach, minor_fluctuation, or sensor_fault. This is what I used to build in realistic failure cases.
- compliance_threshold_c: the safe temperature limit for that product, 4°C for meat, dairy and poultry, 2°C for seafood.
- max_temperature_c: the hottest reading recorded during the 48-hour journey.

- excursion_minutes: how long the shipment spent above its safe temperature.
- door_events: how many times the vehicle door opened.
- final_freshness_index: a quality score from 1.0 (fresh) to 0.0 (spoiled) at the end of the journey.
- risk_label and duration_hours: the overall status (Low, Medium, High, or Data Quality Issue) and the journey length, always 48 hours here.

File 2: sensor_timeseries.csv, the raw sensor readings

This is the detailed version behind the summary table: one row for every 15-minute reading, for every shipment. With 42 shipments over 48 hours, that works out to 8,064 rows in total.

- shipment_id, timestamp, and step: which shipment, what time, and which 15-minute interval it is.
- temperature_c, humidity_pct: the two core IoT readings.
- door_open: 1 if the door was open during that interval, 0 if not.
- vibration_g: a simple stand-in for rough handling or a bumpy road.
- freshness_index: the same quality score as above, but tracked continuously through the journey rather than just at the end.
- band_430nm through band_900nm: 8 columns holding the continuous point-sensor spectral reading described in the dashboard. This is not imaging, just one sensor's reflectance value at each of the 8 wavelengths, every 15 minutes.

File 3: hyperspectral_images.json, the imaging data

This is the file behind the imaging viewer in the dashboard. Rather than covering all 42 shipments, it focuses on 4 example shipments, one that ended up High status, one Medium, one Low (fully compliant), and one with a sensor fault, so the viewer can show a clear contrast.

- For each of those 4 shipments, there are 4 checkpoint images: Farm/Origin Dispatch, Processing Facility, In-Transit Spot Check, and Retail/Arrival Goods-In.
- Each checkpoint image is a 20×20 grid, so 400 pixels, and each pixel has its own Quality Index between 0 and 1.
- For every checkpoint, the file also stores the defect percentage (how much of the image is flagged), the mean quality, and two average spectral curves across the 8 bands, one for normal pixels and one for flagged pixels, which is what powers the signature chart next to the image.

File 4: dashboard_data.json, the numbers behind the dashboard

This is the smallest file, and it's built from the two CSVs. It holds everything the dashboard needs to draw itself without recalculating anything in the browser: the KPI numbers, the alerts list, the full shipment monitoring table, the breakdowns by product and by SME, and the hour-by-hour fleet compliance trend.

> *If asked "how big is the dataset" in the interview: 6 SMEs, 42 shipments, 48-hour journeys, readings every 15 minutes (8,064 sensor rows in total), plus 4 checkpoint hyperspectral images for 4 example shipments (400 pixels and 8 spectral bands each).*

4. The four steps in the pipeline

The demo is built as four small, separate steps. Keeping them separate matches how a real pilot project would work: each step produces an output that could be checked, documented, and handed to someone else.

Step 1: Generate the sensor data (generate_data.py)

This creates the raw 15-minute sensor readings for every shipment: temperature, humidity, door events, vibration. It also calculates a "Freshness Index," a simple number from 1.0 (fresh) down to 0.0 (spoiled) that drops faster the longer a product sits above its safe temperature. This gives every shipment a realistic, gradually-changing quality signal, not just a random one.

Step 2: Generate the hyperspectral images (generate_hyperspectral_images.py)

This is the part that actually creates “images,” not just a single sensor number. At each of the 4 checkpoints, it generates a 20×20 grid of pixels, like a simple photo, and every pixel has its own 8-band spectral reading (400 to 900nm, the same range real hyperspectral cameras use for food inspection). From those 8 bands, each pixel gets a Quality Index using a band-ratio calculation, which is the standard, well-established technique used in real hyperspectral food-quality research.

For a shipment with a cold-chain breach, a patch of degraded pixels appears near one edge of the image after the breach happens, and it grows larger at each later checkpoint. This represents spoilage spreading across the surface of the product. A simple rule then marks any pixel below a quality threshold as “flagged.” This is a basic computer-vision step (thresholding), not a machine-learning model.

Step 3: Monitor and generate alerts (monitoring_analysis.py)

This step takes the raw 15-minute readings and groups them into 1-hour windows (this is called “downsampling,” it's what you'd realistically look at in a report, rather than every single 15-minute reading). It then compares each window against the compliance temperature for that product type (e.g. 4°C for dairy and meat, 2°C for seafood) and works out a status for each shipment: Low, Medium, High, or Data Quality Issue.

Importantly, this step does not use machine learning or prediction. It uses clear, fixed rules based on the thresholds. That was a deliberate choice: this stage of the project is about monitoring and reporting, and rule-based logic is transparent and easy to justify to an auditor or a food-safety regulator, which matters a lot in a POAO context.

Step 4: Build the dashboard (build_dashboard.py)

This step takes everything from Steps 1 to 3 and turns it into a single, self-contained interactive web page (the dashboard) that opens in any browser, no installation needed. It's what I'll actually show in the interview.

5. Walking through the dashboard

Here's what each part of the dashboard shows, and a simple way to describe it out loud.

Dashboard section	What it shows	What to say
Header + KPI cards	Quick numbers: how many shipments, how many SMEs, compliance rate, how many are flagged High status, how many active alerts.	“This is the summary view a project manager would glance at first thing in the morning.”
Pipeline note banner	The 5 steps data goes through, from sensor capture to reporting.	“This shows the workflow end-to-end, which maps directly onto the pilot delivery duties in the job description.”
Temperature & Freshness Index chart	One shipment's temperature over 48 hours, with the compliance line, and how its Freshness Index drops during a breach.	“You can see the exact moment the fridge failed, and how quality dropped afterwards. That's the story I'd put in an incident report.”
In-Transit Spectral Sensor Trend	The same shipment's 8-band sensor reading over time, from one single sensor point, tracked continuously.	“This is the continuous sensor feed, separate from the imaging camera.”
Hyperspectral Imaging viewer	Pick a shipment and a checkpoint, see the actual pixel-grid image, the % of the surface flagged as degraded, and the spectral signature of normal vs. flagged pixels.	“This is the real imaging layer, a picture of the product at each hand-off point, not just a single number.”
Breach % by SME	Which SMEs had the most shipments running above the compliance threshold.	“This is the kind of chart I'd bring to the innovation hub session with an SME to show them where to focus.”

Dashboard section	What it shows	What to say
Product breakdown	How many shipments of each product type (beef, dairy, poultry, lamb, seafood).	“Just context on the mix of the pilot.”
Monitoring status breakdown	How many shipments are Low / Medium / High / Data Quality Issue.	“A one-glance health check across the whole pilot fleet.”
Fleet-wide compliance trend	Compliance rate and average freshness, hour by hour, across every shipment.	“This is what I'd put in a quarterly report to show the trend over time, not just a snapshot.”
Alerts feed	A live list of flagged shipments with a severity level and a recommended action.	“This is what would actually get sent to the SME and logged in the risk register.”
Shipment monitoring register	Every shipment, sortable, with its status. The full underlying table.	“This is the detail behind the summary, the actual register a QA person could work from.”

6. How this connects to the job description

A few direct links worth making out loud in the interview:

- The JD asks for support with “planning, setup, installation, configuration, testing, troubleshooting, and monitoring of pilot technologies.” This demo is a small, self-contained example of exactly that: sensors, imaging, thresholds, and a dashboard, all working together.
- The JD asks for “technical and operational documentation.” This document, and the code comments in each script, are examples of the kind of clear documentation I'd produce for real pilot installations.
- The JD asks for support with “risk register” and “reporting deliverables.” The alerts feed and the shipment monitoring register are directly modelled on that.
- My PhD work on multimodal sensor data (smart-home anomaly detection using temperature, humidity and motion sensors) gave me the same instincts I used here: downsampling, setting sensible thresholds, and evaluating results carefully, just applied to a food supply chain instead of a home.

> *Keep this section honest and light-touch in conversation. The aim is to show relevant thinking, not to overclaim that a demo built in a day is equivalent to production research.*

8. One-line summary if you're short on time

“I built a small synthetic demo of a POAO cold-chain pilot, using IoT sensors plus hyperspectral imaging at checkpoints, with simple, transparent threshold-based monitoring, alerts, and a risk register, to show how I'd approach the technical delivery and reporting side of this role.”